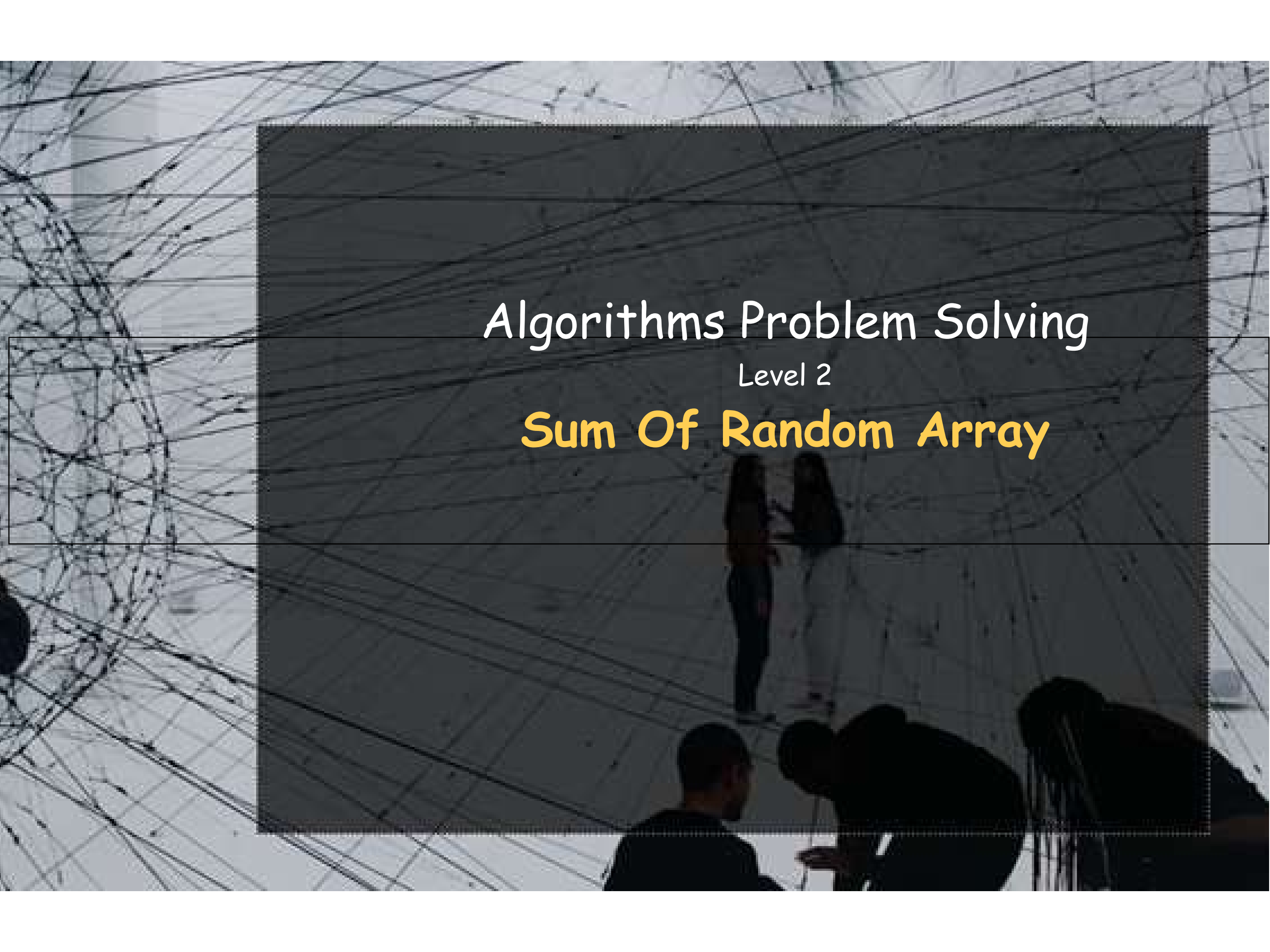


Algorithms Problem Solving

Level 2

Sum Of Random Array

The background of the slide is a photograph of a modern building's interior. It features a large, curved glass ceiling that allows natural light to filter through, creating a pattern of light and shadow on the floor. In the lower portion of the image, the silhouettes of several people are visible, some standing and others sitting, suggesting a collaborative workspace or a public area. The overall color palette is muted, with greys, blues, and the warm tones of the light coming through the glass.

Algorithms Problem Solving

Level 2

Sum Of Random Array

Definition

- ◆ This program fills an array with random numbers from `1` to `100`.
- ◆ It then calculates the sum of all elements in the array.
- ◆ The array can contain up to `100` elements.

Explanation

- ◆ ``FillArrayWithRandomNumber()`` asks the user for the number of elements and fills the array with random numbers from ``1`` to ``100``.
- ◆ ``PrintArrayElements()`` prints all elements stored in the array.
- ◆ ``SumAllNumbInArray()`` is intended to calculate the sum of all array elements.
- ◆ The correct idea for calculating the sum is to start with ``sum = 0`` and add every element to it.
- ◆ In the given code, ``sum`` is initialized with ``arr[0]``, which is incorrect for a general sum.
- ◆ The condition ``if (arr[i] < sum)`` is also incorrect because it only adds elements smaller than the current sum.
- ◆ Therefore, the function does not correctly calculate the sum shown in the example.

Steps in main()

- 1 ☐ Ask the user for the number of array elements.
- 2 ☐ Fill the array with random numbers from `1` to `100`.
- 3 ☐ Print all array elements.
- 4 ☐ Initialize `sum` to `0`.
- 5 ☐ Loop through all array elements.
- 6 ☐ Add each element to `sum`.
- 7 ☐ Return and print the final sum.

💡 Example

◆ Number of elements: `10`

◆ Array:

42 68 35 1 70 25 79 59 63 65

◆ Calculate:

$42 + 68 + 35 + 1 + 70 + 25 + 79 + 59 + 63 + 65$

➡ Result

Array Elements: 42 68 35 1 70 25 79 59 63 65

Sum of all Numbers is: 507

◆ The given comment says `439`, but the actual sum of these numbers is `507`.

📌 Note

- ◆ The correct `SumAllNumbInArray()` function should be:

```
int SumAllNumbInArray(int arr[], int length)
{
    int sum = 0;
    for (int i = 0; i < length; i++)
    {
        sum += arr[i];
    }
    return sum;
}
```

- ◆ `sum += arr[i]` means:

$sum = sum + arr[i]$

- ◆ Unlike finding a maximum or minimum, we do not need a comparison condition when calculating a sum.

⊘ Warning

- ◆ The current function is incorrect:

```
int sum = arr[0];
for (int i = 0; i < length; i++)
{
    if (arr[i] < sum)
        sum += arr[i];
}
```

- ◆ It does not add every element.
- ◆ It also starts the sum with `arr[0]`, which can cause the first element to be counted incorrectly.
- ◆ The comments above `FillArrayWithRandomNumber()` and `SumAllNumbInArray()` are also incorrect/inconsistent with what the functions actually do.
- ◆ `ReadArrayElements()` is defined but never used.
- ◆ `cmath` and `corecrt\_math\_defines.h` are not needed for this program.
- ◆ The array length should ideally be validated so it does not exceed `100`.
- ◆ `rand()` is not seeded with `srand()`, so the program may produce the same pseudo-random sequence each time it runs.



Comparison

▣ `SumAllNumbInArray()`

- ✓ Calculates the total of all array elements.
- ✓ Starts with `sum = 0`.
- ✓ Adds every element using `sum += arr[i]`.
- ✓ Example: `42 68 35 1` → `146`.

▤ `MaxNumbInArray()`

- ✓ Searches for the largest value.
- ✓ Uses `arr[i] > max` to update the maximum.
- ✓ Example: `42 68 35 1` → `68`.

✦ Summary

- ✓ `FillArrayWithRandomNumber()` fills the array with random numbers from `1` to `100`.
- ✓ `PrintArrayElements()` displays the array elements.
- ✓ `SumAllNumbInArray()` should add every element to calculate the total.
- ✓ Initialize the sum with `0`.
- ✓ Use `sum += arr[i]` inside the loop.

Thank you 😊